

Teaching an Agent to Play Super Mario Bros: Deep Q-Learning, Results, and Learning vs. Memorization

Nick Nojiri | September 16, 2026 | Code: github.com/NickNojiri/mario-rl

Summary

I trained a Double Deep Q-Network (DQN) to play level 1-1 of Super Mario Bros from raw screen pixels, using only a CPU. After 3 million decisions (about 9.7 hours), the agent travels **1905** units on average in a controlled evaluation, **2.7x** farther than a random policy (712). It reached the flag 280 times during training. The evidence points to a policy that reacts to the screen rather than replaying a memorized button sequence: it keeps that lead across randomized start times that shift every enemy's position. It did **not** learn a general Mario skill: on level 1-2, which it never trained on, it performs the same as random (386 vs 358). Half of the evaluation episodes on 1-1 stalled. In every stalled episode inspected, the jump button was being held, which points to a partial-observability problem: the screen does not show that the button is already held.

1. The learning problem

Reinforcement learning (RL) is learning by trial and error. The agent repeatedly observes the game, picks a button, and receives a reward. Its goal is a strategy (a *policy*) that maximizes total future reward. Formally this is a Markov Decision Process:

Concept	Plain meaning	In this project
State	What the agent observes	Last 4 frames, grayscale, 84x84 pixels (uint8)
Action	A button choice	5 options: no-op, right, right+jump, right+run, right+jump+run
Reward	Score for the last move	Rightward progress, minus a clock penalty, minus 15 for dying
Episode	One attempt	Start of 1-1 until death, flag, or a no-progress cutoff
Policy	The learned strategy	Pick the button with the highest predicted Q-value

2. ML and RL topics applied

Each technique below is in the codebase. For each: what it is, and why this project needed it.

Topic	What it is, simply	How it was used / why
Q-learning	$Q(\text{state}, \text{action}) = \text{expected total future reward for pressing a button now}$	The agent picks the highest-Q button; learning means making Q accurate.
Temporal-difference (TD) learning	Improve a guess using a slightly better guess one step later	$\text{Target} = r + \gamma * (1 - \text{terminated}) * Q_{\text{target}}(s', a')$. No need to wait for an episode to end.
Discount factor (gamma)	Future reward counts a little less than immediate reward	$\gamma = 0.99$ looks ~100 steps ahead. The tutorial's 0.9 looked ~10 steps, too short to plan a jump.
Terminated vs. truncated	A true ending vs. an episode we cut short	Death and flag zero out the future. The no-progress cutoff keeps bootstrapping, because the game would continue.

Topic	What it is, simply	How it was used / why
Convolutional neural network	Layers that detect edges, then shapes, then objects	Nature-DQN CNN (3 conv + 2 dense) maps 4 frames to 5 Q-values. It is never told what a goomba is.
Frame stacking	Several frames in a row expose motion	One frame cannot show whether Mario is rising or falling.
Experience replay	Store past moves; train on random mini-batches	Consecutive frames are highly correlated. Shuffled batches of 32 from a 100k buffer stabilize learning.
Target network	A frozen copy of the network computes targets	Synced every 10k steps so the goal does not move with every update.
Double DQN	One network picks the next action, the other scores it	Reduces the overestimation caused by always taking a max.
Huber loss + gradient clipping	Limit how much one surprising sample can change the weights	Prevents rare large TD errors from destabilizing training (grad norm capped at 10).
Reward scaling	Rescale rewards to a range the loss handles well	Summed rewards reach +/-60 per step; dividing by 15 keeps TD errors in Huber's useful range.
Adam optimizer	Gradient descent with a per-weight adaptive step size	Its internal statistics are saved in checkpoints, so resumed training behaves identically.
Epsilon-greedy exploration	Act randomly with probability epsilon	Linear decay 1.0 -> 0.1 over 1M steps. The tutorial's multiplicative decay needed ~9.2M steps.
Generalization vs. overfitting	Skill that transfers vs. memorizing the training data	Measured with randomized evaluation and a held-out level (Sections 4-5).
Partial observability / Markov property	The state must contain everything needed to decide	The main failure mode found: frames do not show that the jump button is held (Section 6).
Baselines and statistical comparison	Compare against chance, with uncertainty	Random-policy evaluation plus standard errors over 30 episodes.

Engineering that made the experiment possible

- **Memory:** storing each 84x84 frame once and rebuilding 4-frame stacks at sample time cut the replay buffer from 22.6 GB (float32 pairs) to **706 MB** with identical information.
- **Correctness:** 14 automated tests. The buffer is checked sample-by-sample against naive storage, and a resumed agent must produce an identical next update. One test caught a real bug: the no-progress counter started from the wrong position.
- **Reproducibility:** fixed seeds, pinned dependencies, config hash 8dd876e13de1, and checkpoints that restore weights, optimizer, step count and every random-number state.
- **Hardware:** CPU-only (AMD Ryzen 7 7800X3D) under WSL Ubuntu, because the GPU is not CUDA-capable.

3. Training run

Setting	Value	Setting	Value
Total agent steps	3,000,000 (12M game frames)	Replay buffer	100,000 transitions
Frame skip	4	Batch size / learn every	32 / 3 steps
Learning rate (Adam)	2.5e-4	Target sync	every 10,000 steps
gamma	0.99	Epsilon schedule	1.0 -> 0.1 over 1M steps
Reward scale	1/15	No-progress cutoff	150 steps without new max x_pos
Episodes	18,891	Wall-clock time	~9.7 hours (~85-125 steps/s)

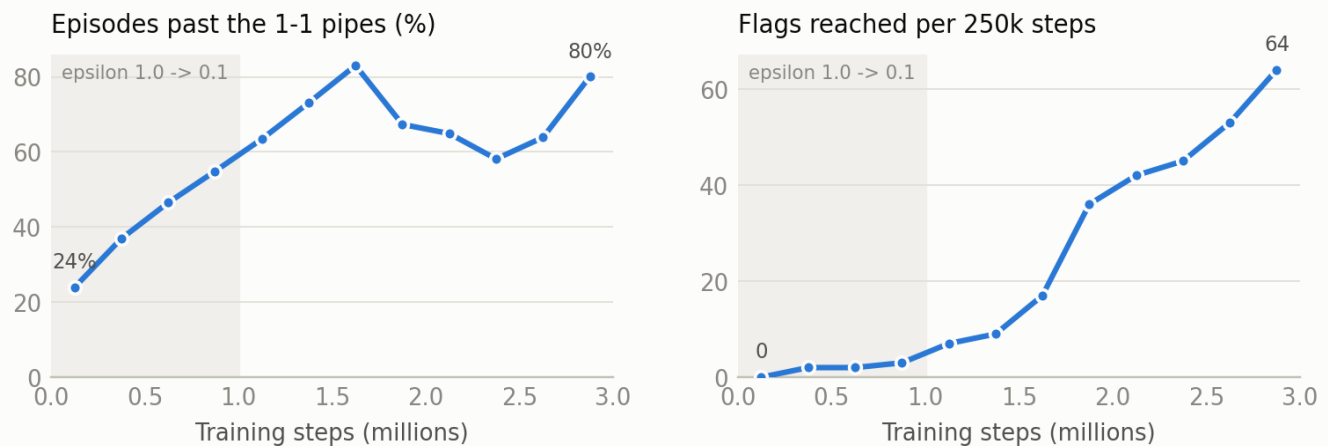


Figure 1. Training progress in 250k-step windows. The shaded region is the exploration phase, where epsilon falls from 1.0 to 0.1. Around x_pos 722 sit the tall pipes that random play almost never clears.

Steps	Epsilon	Episodes	Mean x_pos	Past pipes	Flags
0.00M-0.25M	0.89	1,594	669	24%	0
0.25M-0.50M	0.66	1,542	773	37%	2
0.50M-0.75M	0.44	1,635	856	46%	2
0.75M-1.00M	0.21	1,619	928	55%	3
1.00M-1.25M	0.10	1,745	1082	63%	7
1.25M-1.50M	0.10	1,728	1225	73%	9
1.50M-1.75M	0.10	1,444	1409	83%	17
1.75M-2.00M	0.10	1,454	1312	67%	36
2.00M-2.25M	0.10	1,641	1195	65%	42
2.25M-2.50M	0.10	1,634	1199	58%	45
2.50M-2.75M	0.10	1,580	1283	64%	53
2.75M-3.00M	0.10	1,274	1587	80%	64

Reading the curve. Progress is clear through 1.75M steps (83% of episodes past the pipes). Performance then dips to 58% around 2.25-2.5M before recovering to 80%, while flags keep rising. Temporary regressions like this are a known property of DQN, which bootstraps from its own estimates. Because only the final checkpoint was

kept, the 1.5-1.75M peak cannot be evaluated separately. The code now saves a snapshot every 250k steps.

4. Evaluation protocol and results

Training scores include 10% random actions, so they understate and blur real skill. A separate, fixed evaluation measures the policy directly:

- **30 episodes** per condition, **epsilon 0.01**, and a distinct seed per episode (reproducible, but not identical).
- **Random start delay:** 0-30 no-op steps before control begins, which shifts enemy positions relative to Mario.
- **Held-out level:** the same agent evaluated on level 1-2, which it never trained on.
- **Random-policy baseline** under the identical protocol, so every number can be compared with chance.
- **Unique-trajectory count:** how many of the 30 episodes used a different button sequence.

Condition	Mean x_pos	Mean reward (+/- SE)	Flag rate	Stalled (cutoff)	Unique runs
1-1, random policy	712	618 +/- 50	0%	23%	-
1-1, trained DQN	1905	1797 +/- 118	3%	50%	30/30
1-2, random policy	358	288 +/- 51	0%	3%	-
1-2, trained DQN (held out)	386	306 +/- 36	0%	50%	28/30
1-1, trained DQN, fully greedy, no random start	898	819 +/- 0	0%	0%	1/30

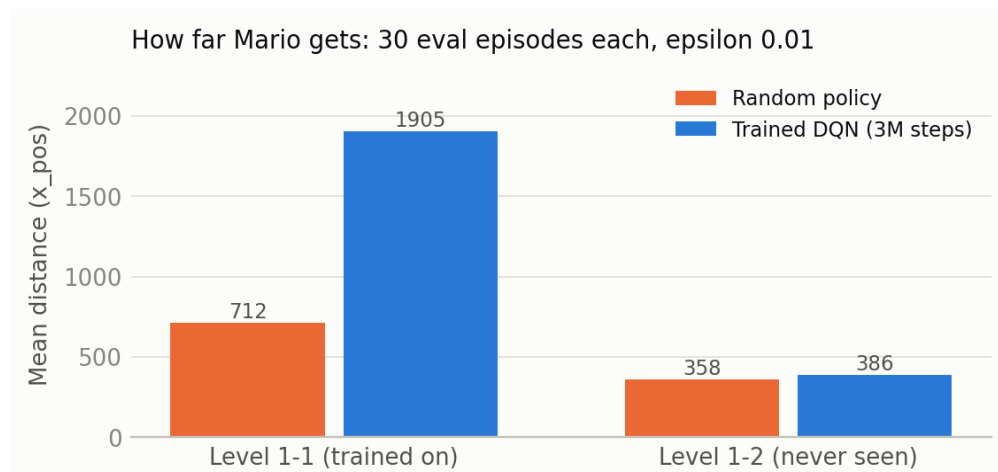


Figure 2. Trained agent vs. random policy. On 1-1 the reward difference is 9.2 standard errors; on held-out 1-2 it is 0.3, which is indistinguishable from chance.

5. Did it learn, or did it memorize?

This is the central question for any agent trained on a single, deterministic level. The NES emulator has no randomness: the same buttons always produce the same outcome. An agent could therefore score well by replaying one memorized button sequence, like memorizing exam answers instead of learning the material. The project was designed to detect and discourage that.

How the setup pushes toward learning

Design choice	Why it discourages memorization
Random 0-30 step start delay (training and eval)	Enemies are in different places relative to Mario each episode, so a fixed tape of button presses breaks. The agent must react to what is on screen.
Epsilon-greedy noise during training (10% after 1M steps)	Random actions knock the agent off any single path, so it must recover from many different situations.
Experience replay over 100k transitions	Each update mixes moments from thousands of episodes rather than rehearsing the latest run.
Pixel input only; no map, route or positions supplied	Nothing tells the agent where to go. It only sees the screen and the score.
Evaluation with distinct seeds and random start offsets	Scores come from many different situations, so a policy that only works along one path cannot score well.
Held-out level 1-2 and a random baseline	Separates skill on 1-1 from general skill, and both from chance.

What the evidence shows

Evidence	Result	Interpretation
Trained vs. random on 1-1, across 31 random start offsets	2.7x farther; reward +9.2 SE	Genuine, statistically clear learning. A fixed memorized sequence would fall apart once enemies shift; this policy keeps its lead, so it is reacting to the screen.
Distinct trajectories on 1-1	30 of 30	Consistent with the above, but weak on its own: different start delays already change the sequence.
Trained vs. random on held-out 1-2	1.08x; reward +0.3 SE	No transfer. The skill is specific to 1-1's layout and look.
Fully greedy, fixed start	1 trajectory, dies at x_pos 898 every time	With all randomness removed, the policy is deterministic and brittle at one spot.

Conclusion. The agent **learned a reactive policy for level 1-1** rather than memorizing a button sequence: it keeps a large, statistically clear lead over chance even when every episode starts with enemies in different places. It did **not** learn to play Mario in general, because it gained nothing on an unseen level. That is the expected outcome of training on a single level: the network only ever saw 1-1, so the features it learned (this pipe, these gaps, this background) do not describe 1-2's underground visuals. Generalization has to be trained for, most directly by training on several levels at once.

6. Failure analysis: why half the evaluation episodes stalled

Fifty percent of trained 1-1 evaluation episodes ended through the no-progress cutoff rather than death or the flag. A per-episode diagnosis of 12 episodes found 1 flag, 7 deaths and 4 stalls. **In all 4 stalls, the jump button (A) was held for 100% of the final 150 steps.**

In Super Mario Bros, A must be released before Mario can jump again. The agent's state is only screen pixels, and pixels do not show whether A is currently held. Two situations that look identical on screen need different

actions ("press A" vs. "release A first"), so the state violates the **Markov property**. This is **partial observability**. Facing a pipe or staircase, the highest-Q action is "jump", but holding jump does nothing, and the agent has no way to perceive why.

During training, the 10% random actions released A by chance, which hid the problem. At evaluation epsilon 0.01, almost nothing does. The planned fix is to feed the previous action into the network alongside the frames, making the button state observable.

7. Limitations and next steps

- **Single training level:** train on several stages (e.g. 1-1 to 1-4) and hold out others to measure real generalization.
- **Partial observability:** add the previous action as a network input; compare against a recurrent (LSTM) variant.
- **One seed, one run:** RL results vary across seeds; repeat with 3+ seeds and report the spread.
- **Checkpoint selection:** evaluate the new 250k-step snapshots to measure the mid-run dip and pick the best policy.
- **Stochasticity:** add sticky actions (repeat the last action with small probability) as a further guard against memorization.

8. Reproducibility

All code, tests and this report's generator are public at <https://github.com/NickNojiri/mario-rl>. Run 1 used commit 90081ba; results and figures come directly from the run's logs.

```
python train.py --preset full --run-name dqn_1_1      # training (3M steps)
python eval.py runs/dqn_1_1/latest.pt --episodes 30  # evaluation on 1-1
python eval.py runs/dqn_1_1/latest.pt --stage 2      # held-out level 1-2
python eval.py runs/dqn_1_1/latest.pt --epsilon 1.0  # random-policy baseline
python -m pytest tests                               # 14 tests
```

Acknowledgement. Software development for this project was done with an AI coding assistant (Claude Code). Experiments were run locally, and all reported numbers come from those runs.